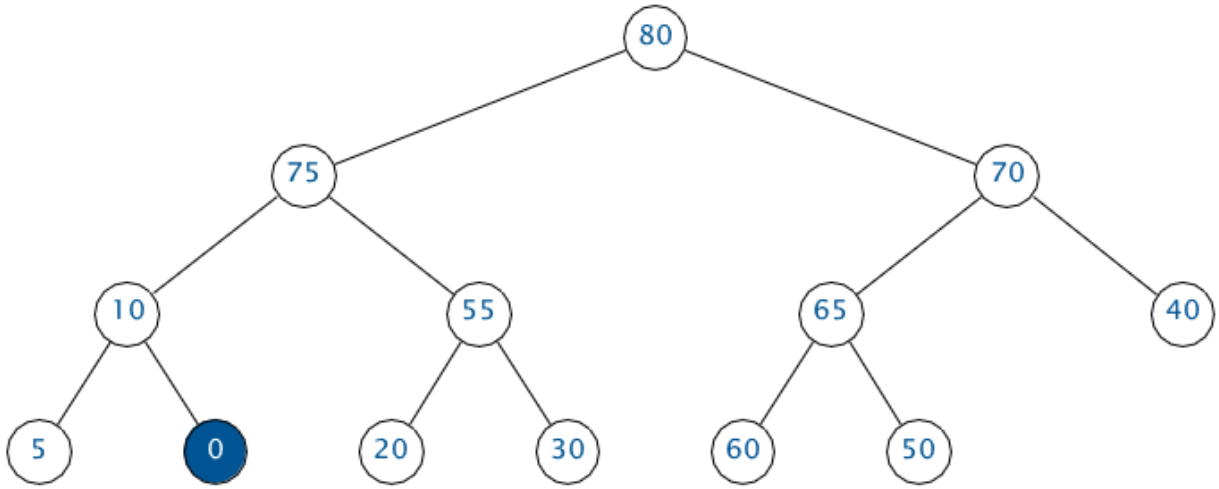
Xem demo heap
Binary Heap (Priority Queue) - VisuAlgo

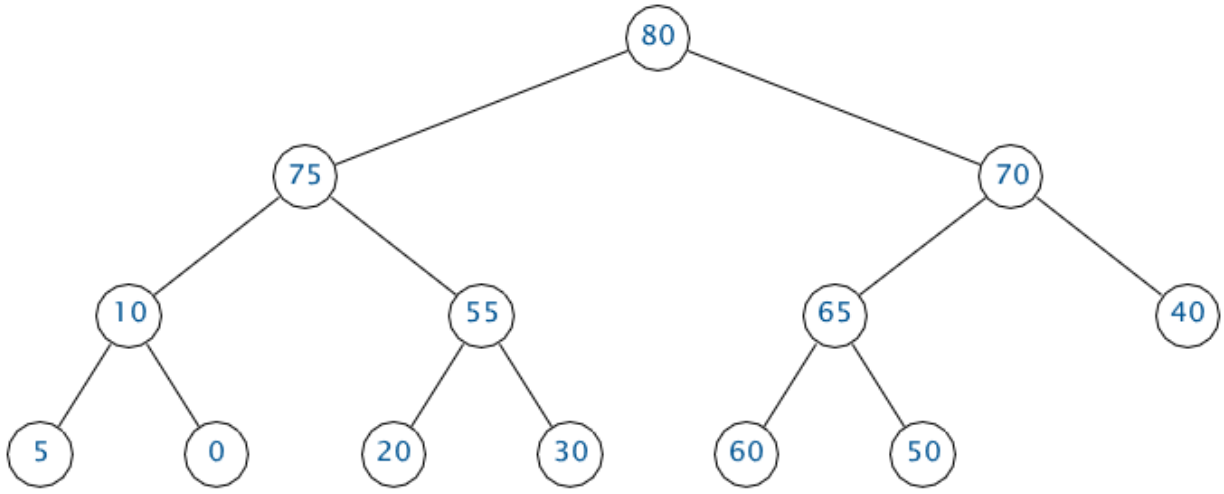
1. COS226, Midterm f25, 2b

Xét biểu diễn max-heap bằng cây nhị phân dưới đây. Giả sử key ở nút mầu xanh được sửa từ 0 thành 77, sau đó hàm swim() được gọi để khôi phục thứ tự heap. Hỏi các cặp key nào được so sánh trong quá trình chạy hàm swim đó?



2. COS226, Midterm s25, 2b

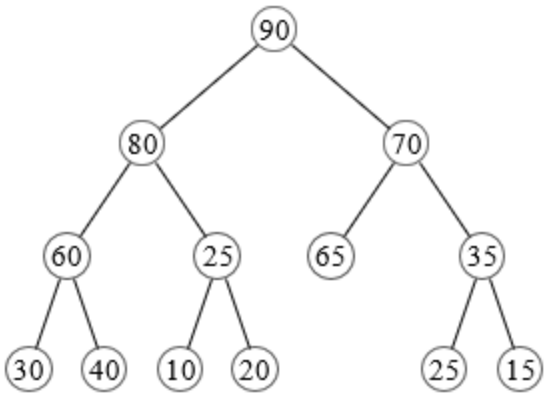
Xét biểu diễn max-heap bằng cây nhị phân dưới đây.



Giả sử thao tác được thực hiện tiếp theo là DELETE-MAX, Hỏi các cặp key nào sẽ được so sánh trong quá trình chạy hàm đó?

3. COS226, Midterm f24, 4b

Dưới đây có thể là biểu diễn cây nhị phân của một heap-max hay không?



4. COS226, Midterm f25, 1b

Giả sử rằng một MinPQ được triển khai bằng cách sử dụng một binary heap với một mảng có thể thay đổi kích thước (tăng gấp đôi khi đầy và giảm một nửa khi đầy ≤ 1/4). Thời gian chạy tồi nhất của đoạn mã sau đây theo hàm của n là bao nhiêu? Điền vào ô phù hợp nhất.

```
MinPQ<Integer> pq = new MinPQ<Integer>();
for (int i = 0; i < n; i++)
        pq.insert(a[i]);
for (int i = 0; i < n; i++)
        a[i] = pq.delMin();
```

◯ $\Theta(\log n)$ ◯ $\Theta(n)$ ◯ $\Theta(n \log n)$ ◯ $\Theta(n^2)$ ◯ $\Theta(n^2 \log n)$

Bài 1.

Lấy mã nguồn chương trình và dữ liệu về các cài đặt (implementation) Hàng đợi ưu tiên (max/min priority queue) và Sắp xếp sử dụng cây thứ tự bộ phận (heap sort) tại link mã nguồn môn hoc. Chạy lại chương trình với dữ liệu test. Chú ý: Thư viện algs4.jar

Bài 2. Java String Reverse

Tóm tắt: Kiểm tra, rồi in ra xem một chuỗi kí tự có phải là “palindrome” hay không ? Chuỗi kí tự là “palindrome” nếu đọc xuôi giống như đọc ngược.

Bài 3. Jesse and Cookies

Tóm tắt: Cho N số nguyên (cookies) và ngưỡng K (độ ngọt).
Bước 1: Nếu số nhỏ nhất trong dãy nhỏ hơn K, lấy số đó ra – giả sử là m1. Sau đó tìm số nhỏ nhất trong các số còn lại – giả sử là m2. Thay số m2 bằng số có giá trị (m1 + 2 * m2).
Bước 2: Lặp lại bước 1 cho tới khi không còn số nào nhỏ hơn K.
Bước 3: Hãy in ra số lần thay số cần làm để tất cả các số còn lại lớn hơn hay bằng K. In ra -1 nếu không thể làm được.
Gợi ý: Đoạn mã sau nhập vào các giá trị N, K, rồi nhập tiếp các số nguyên (cookies) vào trong một hàng đợi ưu tiên (min priority queue – giúp lấy được giá trị nhỏ nhất). (/* Sau đó in ra các số theo thứ tự tăng dần. */)

```
Scanner scanner = new Scanner(System.in);
int N, K;
N = scanner.nextInt();
K = scanner.nextInt();

Queue pqueue = new PriorityQueue(N);
for (int i = 0; i < N; i++) {
    pqueue.add(scanner.nextInt());
}

/*
while (pqueue.size() != 0) {
    System.out.println(pqueue.remove());
}
*/
```

Link: How do I use a PriorityQueue ? trên StackOverFlow

Bài 4 (không bắt buộc) Java Priority Queue

Tóm tắt: Các sinh viên sắp hàng, được phục vụ theo trật tự ưu tiên, lần lượt là: a) Điểm (CGPA) càng cao càng tốt, b) tên theo trật tự chữ cái và c) mã (token – id) càng nhỏ càng tốt.

Nếu lệnh là “ENTER” thì đưa sinh viên vào hàng; nếu là “SERVED” thì sinh viên được ưu tiên nhất (được phục vụ và) ra khỏi hàng.
Yêu cầu: In ra các sinh viên còn lại trong hàng, theo trật tự ưu tiên (cao xuống thấp). Nếu hàng rỗng, in ra “EMPTY”.
Gợi ý: Xem link: How do I use a PriorityQueue ? trên StackOverFlow. Tham khảo bài Java Sort tuần trước. Trước tiên, cần tạo lớp giúp so sánh hai sinh viên, dạng:

```
class StudentComparator implements Comparator  {
      @Override
      public int compare(Student x, Student y)
      {
            //Complete your code

            return 0;
      }
}
```

Chú ý: Trong Java, so sánh xem String nào lớn hơn, dùng phương thức compareTo (vd: str1.compareTo(str2)).
Sau đó, trong chương trình chính, tạo priority queue sử dụng lớp so sánh đã tạo ở trên. Ví dụ:

```
      Comparator comparator = new StudentComparator();
      Queue pq = new PriorityQueue(10, comparator);
```

Bài 5. Find the Running Median

Tóm tắt: Đọc lần lượt vào N số nguyên. Sau mỗi lần đọc vào số thứ i, in ra giá trị trung vị của i số đã nhập vào.
Với dãy có (2k + 1) số, thì giá trị trung vị bằng phần tử đứng giữa (có k số nhỏ hơn (hoặc bằng) nó, và đồng thời có k số lớn hơn (hoặc bằng) nó). Với dãy có (2k) phần tử, giá trị trung vị bằng trung bình cộng hai phần tử đứng giữa.
Gợi ý: Chia dãy đang được nhập vào làm 2 phần có số phần tử tương đương, và lưu tương ứng vào 2 hàng đợi ưu tiên.
Đoạn mã sau giúp khai báo 2 hàng đợi ưu tiên chứa các số nguyên; một hàng đợi là min priority queue giúp có được phần tử nhỏ nhất và một hàng đợi là max priority queue giúp có được phần tử lớn nhất.

```
    Scanner scanner = new Scanner(System.in);
    int N;
    N = scanner.nextInt();
    Queue maxPQ = new PriorityQueue(N/2 + 1, Collections.reverseOrder());
    Queue minPQ = new PriorityQueue(N/2 + 1);
```

Giả sử priorityQueue là một hàng đợi ưu tiên các số nguyên. Lệnh sau đọc (nhưng không xoá đi) phần tử đầu hàng đợi, in ra dưới dạng (format) có một số thập phân sau dấu phẩy.

```
   System.out.printf(“%.1f\n”, (double) priorityQueue.peek());
```

Hãy phân tích độ phức tạp tích toán của thuật toán.

Bài 6. Đếm từ (Bảng tra cứu)

Viết một chương trình đọc một chuỗi văn bản từ input chuẩn (hoặc file tùy bạn) đếm số lần xuất hiện của mỗi từ trong văn bản, in ra kết quả là danh sách các từ theo thứ tự từ điển kèm theo số lần xuất hiện. Hãy viết mà không dùng các cấu trúc HashMap hoặc TreeMap sẵn có trong thư viện của java.
Sau khi chương trình chạy đúng với dữ liệu nhỏ, hãy thử với dữ liệu văn bản lớn. Nếu chương trình của bạn chạy chậm, hãy nghĩ cách cải tiến cấu trúc dữ liệu và giải thuật của bạn.
Sau khi cải tiến xong hoặc không cải tiến được, bạn hãy thử dùng TreeMap.

2.4.1 Giả sử chuỗi sau được áp dụng cho một hàng đợi ưu tiên ban đầu rỗng

`P R I O * R * * I * T * Y * * * Q U E * * * U * E`

(trong đó các chữ cái có nghĩa nó được thêm vào hàng đợi và dấu * có nghĩa xóa key lớn nhất. Cho biết chuỗi kết quả thu được từ các lệnh xóa key lớn nhất.

2.4.2 Hãy phản biện ý tưởng sau:
Để cài hàm tìm max của một dãy chạy với thời gian hằng số, tại sao không dùng một cái stack hay queue và lưu giữ giá trị lớn nhất đã được thêm vào từ đầu cho đến giờ, sau đó trả về giá trị đó làm đáp án cho câu hỏi tìm giá trị lớn nhất?

2.4.4 Hỏi một mảng được sắp xếp giảm dần có biểu diễn một heap loại max không??

2.4.5 Hãy vẽ heap thu được khi thêm các khóa sau vào một heap max ban đầu rỗng theo đúng thứ tự đã cho `E A S Y Q U E S T I O N .`

2.4.6 Dùng cách ký hiệu tương tự bài 2.4.1. Hãy cho biết chuỗi các heap thu được khi thực hiện chuỗi sau

`P R I O * R * * I * T * Y * * * Q U E * * * U * E`

đối với một heap max ban đầu rỗng.

2.4.7 Phần tử lớn nhất của một heap phải nằm tại vị trí số 1, phần tử lớn nhìn phải nằm tại vị trí số 2 hoặc 3. Hãy cho biết các vị trí tại một heap kích thước 31 mà key lớn thứ k
(i) có thể xuất hiện
(ii) không thể xuất hiện
Với k = 2,3,4 (giả sử các key trong heap đều có giá trị khác nhau).

2.4.8 Tương tự bài trên nhưng trả lời cho key nhỏ thứ k.

2.4.9 Vẽ tất cả các heap khác nhau có thể tạo từ 05 key `A B C D E`,
Sau đó vẽ tất cả các heap khác nhau có thể tạo từ 05 key `A A A B B`.

2.4.10 Giả sử ta muốn tiết kiệm ô đầu tiên trong heap dạng mảng `pq[]`, ta muốn đặt giá trị lớn nhất ở ô `pq[0]`, các nút con của nó tại `pq[1]` và `pq[2]`, và cứ tiếp tục như vậy. Hỏi nút cha và các nút con của `pq[k]` nằm ở đâu?

2.4.11 (*) Giả sử ứng dụng của bạn sẽ có rất nhiều lệnh gọi insert, nhưng chỉ có vài lệnh xóa delMax. Cách cài đặt nào là hiệu quả nhất cho hàng đợi ưu tiên? heap, mảng không sắp thứ tự, hay mảng có sắp thứ tự?

2.4.12 (*) Giả sử ứng dụng của bạn sẽ có rất nhiều lệnh tìm max, nhưng chỉ có vài lệnh gọi insert và remove. Cách cài đặt nào là hiệu quả nhất cho hàng đợi ưu tiên? heap, mảng không sắp thứ tự, hay mảng có sắp thứ tự?

2.4.13 (*) Hãy mô tả một cách tránh phải kiểm tra `j < N` tại hàm `sink()`.

2.4.14 (*) Trong quá trình delMax xóa phần tử lớn nhất khỏi một heap kích thước N với các key phân biệt. Hỏi ít nhất phải đổi chỗ bao nhiêu phần tử? Hãy cho ví dụ đạt được số phép đổi chỗ tối thiểu của

heap kích thước 15.
Trả lời câu hỏi tương tự đối với hai phép delMax liên tiếp, ba phép delMax liên tiếp.

2.4.15 (*) Hãy thiết kế một thuật toán đảm bảo thời gian tuyến tính để kiểm tra xem một mảng `pq[]` có phải là một heap min hay không.

2.4.16 (*) Với N=32, hãy liệt kê các mảng gồm N phần tử mà heapsort cần sử dụng số phép so sánh ít nhất có thể. Và nhiều nhất có thể.

2.4.17 (*) Giả sử ta xây dựng một hàng đợi ưu tiên loại min kích thước k, sau đó thực hiện N-k lần phép thay thế phần tử min (chèn thêm 1 phần tử sau đó xóa min). Chứng minh rằng kết quả thu được sau đó là hàng đợi ưu tiên chứa k số lớn nhất của N phần tử.

2.4.18 (*) Trong `MaxPQ`, giả sử một chương trình client gọi hàm `insert()` để thêm vào một phần tử lớn hơn tất cả các phần tử hiện có trong hàng đợi, và ngay sau đó gọi `delMax()`. Giả sử rằng tất cả các key phân biệt. Hỏi rằng heap thu được có giống hệt trạng thái của heap trước khi thực hiện lệnh insert trên kia hay không?
Trả lời câu hỏi tương tự với hai phép `insert()` với lần thứ nhất thêm một số lớn hơn tất cả các số hiện trong queue, lần hai thêm một số còn lớn hơn số trước, sau đó là hai lệnh `delMax()`.

2.4.19 (*) Cài constructor cho `MaxPQ` với tham số là một mảng các phần tử và sử dụng phương pháp xây dựng heap theo kiểu bottom-up.

2.4.20 (*) Chứng minh rằng quy trình xây dựng heap bằng việc sử dụng các lệnh sink sử dụng đến ít hơn 2N phép so sánh và ít hơn N phép đổi chỗ.